

ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA EN SISTEMAS



SISTEMA DE RUTAS INTELIGENTES PARA TRANSPORTE

INTEGRANTES

Nicole Achote

Gary Defas

Anthony Alangasí

Lindsay Guzmán

20 de mayo de 2026



1. Descripción del problema y motivación de la aplicación

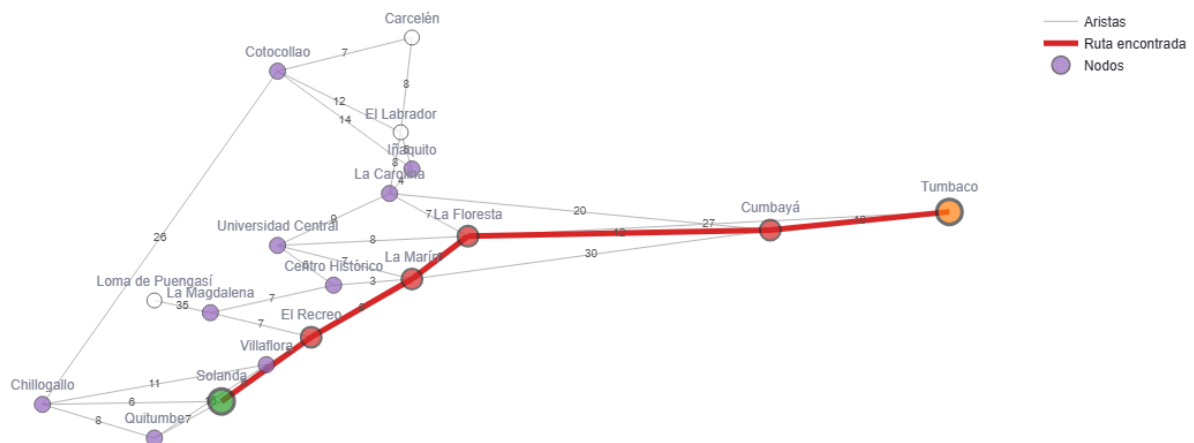
Los sistemas de transporte urbano presentan problemas relacionados con tráfico, tiempo de desplazamiento y planificación eficiente de rutas. En ciudades como Quito, el costo de movilizarse entre sectores varía según la distancia, congestión vehicular y dificultad de circulación.

Este proyecto aborda el problema de búsqueda de rutas óptimas mediante un grafo ponderado dirigido utilizando algoritmos de Inteligencia Artificial como UCS, GBFS y A*. En el sistema, los nodos representan sectores estratégicos de la ciudad y las aristas representan conexiones viales con costos asociados.

La motivación del proyecto es comparar el comportamiento de distintos algoritmos de búsqueda y demostrar que la ruta aparentemente más cercana no siempre es la más eficiente, permitiendo analizar las ventajas y limitaciones de cada algoritmo en escenarios de movilidad urbana.

2. Representación del problema

Grafo base de Quito - ruta resaltada de A*



El problema se representa mediante un grafo ponderado dirigido, diseñado para simular un sistema inteligente de rutas urbanas aplicado al transporte y entregas dentro de sectores importantes de Quito y sus valles cercanos.



Los nodos representan sectores, barrios, paradas importantes y zonas estratégicas de movilidad urbana. Cada nodo simboliza un punto relevante dentro de la red de transporte de la ciudad.

Entre los nodos utilizados se encuentran:

Quitumbe, Chillogallo, Solanda, Villaflora, El Recreo, La Magdalena, Centro Histórico, La Marín, Universidad Central, La Floresta, La Carolina, Iñaquito, El Labrador, Cotocollao, Carcelén, Cumbayá, Tumbaco

Cada nodo posee coordenadas ficticias coherentes con su posición relativa dentro de la ciudad. Estas coordenadas no representan ubicaciones exactas, sino una aproximación

Tipo de grafo

El sistema utiliza un grafo dirigido y ponderado.

- Es dirigido porque algunas rutas pueden tener costos diferentes dependiendo del sentido de circulación, simulando condiciones reales como tráfico, semáforos, o congestión vial.
- Es ponderado porque cada arista posee un costo asociado que representa el esfuerzo necesario para desplazarse entre dos sectores.

Las aristas representan las conexiones viales entre nodos.

Definición del costo

Los pesos asignados a las aristas no representan únicamente distancia física exacta. En este proyecto se utiliza un costo definido como:

$$\text{Costo} = \text{distancia} + \text{tráfico}$$

La definición del costo busca simular de manera más realista el comportamiento del transporte urbano, donde una ruta más corta puede resultar menos eficiente debido al tráfico o dificultad de circulación.

Heurística utilizada

La heurística implementada se basa en la distancia euclidiana entre las coordenadas ficticias de los nodos:

$$h(n) = \text{distancia_euclidiana}(n, \text{meta}) \times 2.0$$



La constante 2.0 se utiliza como factor de escala para mantener la heurística conservadora dentro del diseño del grafo.

La heurística cumple dos propiedades fundamentales:

- **Admisibilidad**

La heurística no sobreestima el costo real mínimo hacia la meta, permitiendo que A^* conserve optimalidad.

- **Consistencia**

La heurística cumple la siguiente condición para cada arista:

$$h(u) \leq c(u,v) + h(v)$$

Esto garantiza estabilidad durante la expansión de nodos y evita reevaluaciones innecesarias.

3. Algoritmos implementados

3.1. UCS (Búsqueda de Costo Uniforme). Es un algoritmo búsqueda no informado, por lo que no tiene información de qué tan cerca o lejos se encuentra de la meta. Este explora el espacio de estados expandiendo los nodos con menor costo acumulado desde el origen y garantiza encontrar la ruta óptima siempre que los costos sean positivos.

3.2. GBFS (Greedy Best-First Search). Es un caso especial de BFS que utiliza una función heurística que permite estimar el costo restante desde el nodo actual hasta el objetivo. Se lo considera un algoritmo codicioso porque únicamente evalúa el costo estimado del nodo que va a expandir, ignorando el costo acumulado para llegar a dicho nodo. Esto le permite encontrar rápidamente la ruta, pero, posiblemente, no sea la ruta óptima.

3.3. A* (A estrella). Este algoritmo combina las ventajas de UCS y GBFS al evaluar los nodos utilizando una combinación de la función heurística más el costo acumulado. Dado que la función heurística que se ha empleado es admisible y consistente, A^* garantiza encontrar la ruta óptima expandiendo menos nodos que UCS.

3.4. Pseudocódigo

Para implementar estos 3 algoritmos se utilizó una función general que encapsula su comportamiento cambiando entre estos a través de cálculo de la prioridad o función de costo.



Función BusquedaPrioridad(grafo, inicio, meta, heuristica, tipo_algoritmo)

// Inicialización

contador <- 0

prioridad_inicial <- CalcularPrioridad(inicio, 0.0, meta, heuristica, tipo_algoritmo)

// Frontera guarda: (prioridad, contador, nodo_actual, g_costo)

frontera <- CrearColaPrioridad()

InsertarEnCola(frontera, (prioridad_inicial, contador, inicio, 0.0))

padres <- DiccionarioVacío()

padres[inicio] <- Nulo

mejor_costo <- DiccionarioVacío()

mejor_costo[inicio] <- 0.0

visitados <- ConjuntoVacío()

Mientras frontera NO esté vacía:

// Selección del nodo con mayor prioridad (menor valor numérico)

(prioridad, _, actual, actual_g) <- ExtraerMínimo(frontera)

Si actual ya está en visitados:

Continuar al siguiente ciclo

Añadir actual a visitados

// Condición de parada (Prueba de meta al expandir)

Si actual == meta:

Retornar ReconstruirRuta(padres, meta), actual_g

// Expansión de sucesores

Para cada (vecino, costo_arista) en Sucesores(grafo, actual):

Si vecino está en visitados:

Continuar

nuevo_g <- actual_g + costo_arista



```
// Relajación de la arista (Evaluación de mejores rutas)
```

```
Si nuevo_g < mejor_costo.obtener(vecino, Infinito):
```

```
    mejor_costo[vecino] <- nuevo_g
```

```
    padres[vecino] <- actual
```

```
    contador <- contador + 1
```

```
nueva_prio <- CalcularPrioridad(vecino, nuevo_g, meta, heuristica, tipo_algoritmo)
```

```
InsertarEnCola(frontera, (nueva_prio, contador, vecino, nuevo_g))
```

```
Retornar "Ruta no encontrada"
```

```
Fin Procedimiento
```

4. Resultados

Para validar el rendimiento y las propiedades teóricas de los algoritmos implementados, se ejecutaron cuatro consultas estratégicas bajo un perfil de tráfico normal. A partir de las pruebas ejecutadas en la interfaz web, se obtuvieron las siguientes métricas de rendimiento para cada uno de los escenarios previstos:

Caso 1: Solanda hasta Tumbaco

Algoritmo	Ruta	Costo total	N.º pasos	Nodos generados	Nodos expandidos	Tiempo (ms)
UCS	Solanda → El Recreo → La Marín → La Floresta → Cumbayá → Tumbaco	48	5	24	17	1.4114
GBFS	Solanda → El Recreo → La Marín → Cumbayá → Tumbaco	58	4	13	5	0.4366
A*	Solanda → El Recreo → La Marín → La Floresta → Cumbayá → Tumbaco	48	5	20	15	0.9787

Caso 2: Quitumbe hasta Tumbaco

Algoritmo	Ruta	Costo total	N.º pasos	Nodos generados	Nodos expandidos	Tiempo (ms)
UCS	Quitumbe → Solanda → El Recreo → La Marín → La Floresta → Cumbayá → Tumbaco	55	6	24	17	2.0863
GBFS	Quitumbe → Villaflores → El Recreo → La Marín → Cumbayá → Tumbaco	69	5	13	6	0.3763
A*	Quitumbe → Solanda → El Recreo → La Marín → La Floresta → Cumbayá → Tumbaco	55	6	22	15	0.8988



Caso 3: Cotacollao hasta Cumbayá

Algoritmo	Ruta	Costo total	N.º pasos	Nodos generados	Nodos expandidos	Tiempo (ms)
UCS	Cotacollao → Iñaquito → La Carolina → La Floresta → Cumbayá	37	4	19	13	1.9924
GBFS	Cotacollao → Iñaquito → La Carolina → Cumbayá	38	3	9	4	0.3854
A*	Cotacollao → Iñaquito → La Carolina → La Floresta → Cumbayá	37	4	14	8	0.8651

Caso 4: Universidad Central hasta El Labrador

Algoritmo	Ruta	Costo total	N.º pasos	Nodos generados	Nodos expandidos	Tiempo (ms)
UCS	Universidad Central → La Carolina → El Labrador	17	2	16	9	1.0043
GBFS	Universidad Central → La Carolina → El Labrador	17	2	8	3	0.3568
A*	Universidad Central → La Carolina → El Labrador	17	2	13	7	0.6723

5. Análisis comparativo

- **Rendimiento**

Tiempo de ejecución: Como se observa en los cuatro casos, GBFS es el más rápido en todos los escenarios, resolviendo los problemas en un rango de 0.35 ms a 0.43 ms. A* ocupa el segundo lugar (0.67 ms a 0.97 ms), superando ampliamente a UCS, que llega a duplicar el tiempo de A*.

Consumo de espacio: UCS presenta un mayor uso de memoria al generar y expandir la mayor cantidad de nodos. A* optimiza significativamente este aspecto al reducir el espacio de búsqueda; en el trayecto largo del Caso 2, encuentra la solución óptima generando 22 nodos y expandiendo solo 15.

- **Calidad de la solución**

Optimalidad del costo: UCS y A* ofrecen una calidad de solución máxima e idéntica, garantizando el camino de menor costo en los cuatro casos presentados.

Longitud de la ruta: Se evidencia que, para minimizar el costo, UCS y A* eligen rutas con más pasos. En cambio, GBFS prioriza rutas con menos pasos,



pero con un costo total significativamente más alto.

- **Ventajas**

UCS: Su principal ventaja es la certeza absoluta de encontrar la ruta de costo mínimo sin necesidad de poseer información geográfica previa o coordenadas de la ciudad.

GBFS: Ofrece un costo computacional mínimo y una velocidad de respuesta ideal para sistemas en tiempo real donde la precisión del costo no sea crítica.

A*: Reúne lo mejor de ambos mundos; provee soluciones matemáticamente óptimas a una velocidad muy cercana a la de un algoritmo ávido, maximizando la eficiencia global del sistema.

- **Limitaciones del Sistema**

UCS: Gasta memoria y tiempo explorando en direcciones que se alejan del destino si los costos acumulados locales se mantienen bajos.

GBFS: Es incapaz de reconocer calles congestionadas, accidentes viales o penalizaciones de costo en el trayecto, lo que lo vuelve inestable para redes de transporte reales.

A*: Su efectividad depende estrictamente de la calidad de la heurística. Si el cálculo de la función $h(n)$ es incorrecto o sobreestima el costo real, el algoritmo pierde su propiedad de optimalidad.

6. Conclusiones y trabajo futuro

Conclusiones

- El desarrollo de este sistema permitió comprender cómo los algoritmos de búsqueda de Inteligencia Artificial pueden aplicarse a problemas reales de movilidad y optimización de rutas. Mediante un grafo ponderado dirigido se representó una red urbana donde los costos consideran distancia, tráfico y dificultad de circulación.
- La comparación entre UCS, GBFS y A* evidenció diferencias importantes: UCS encuentra la ruta de menor costo, GBFS puede equivocarse al usar solo la heurística y A* ofrece el mejor equilibrio entre eficiencia y optimalidad.



- Además, se comprobó la importancia de utilizar una heurística admisible y consistente para mejorar el rendimiento del sistema.

Trabajo futuro

- Como trabajo futuro, el sistema podría incorporar tráfico en tiempo real, mapas interactivos con herramientas como OpenStreetMap y Folium, además de restricciones dinámicas como accidentes o cierres viales. También podría evolucionar hacia una aplicación web o móvil orientada a transporte y movilidad inteligente en ciudades como Quito.